

DATA 581

Modeling and Simulation II



Markov Chains Part 2

Outline

- Reversible Markov Chains
- The Metropolis-Hastings Algorithm
- A First Look at Bayesian Statistics
- A Worked Example
- Built-in Software and Random Walk Metropolis-Hastings
- Application to Mean Prediction

In what follows, the state space can be infinite or finite.

```
set.seed(222696) # use this to reproduce output
```

Reversible Markov Chains

A Markov chain is said to be *time-reversible* if the Markov property holds for the chain when it is time reversed:

$$P(X_n = x_n | X_{n+1} = x_{n+1}, \dots) = P(X_n = x_n | X_{n+1} = x_{n+1})$$

Theorem

A Markov Chain with transition matrix P is reversible if there exists a vector q such that

$$q_j P_{ji} = q_i P_{ij}.$$

Reversible Markov Chains

For such a $\mathbf{q}$, observe what happens when we multiply $\mathbf{q}$ by P .

The i th component of the vector $\mathbf{q}P$ is

$$\sum_{j=1}^{\infty} q_j P_{ji}$$

and this is

$$q_i \sum_{j=1}^{\infty} P_{ij} = q_i$$

if the Markov chain is reversible.

Therefore

$$\mathbf{q}P = \mathbf{q}.$$

$\leadsto$ $\mathbf{q}$ is the steady state vector for P .

Reversible Markov Chains

Example: Symmetric Random Walk

Suppose $S = \{0, \pm 1, \pm 2, \dots, \pm k\}$ **for some** $k > 2$.

$$X_n = X_{n-1} + 2B_n - 1$$

where B_n **is Bernoulli with parameter** $p = .5$, **independent of** X_{n-1} .

When $X_{n-1} = \pm k$, $X_n = k - 1$ **(or** $1 - k$ **)**.

For $|j| < k$,

$$P_{j,j+1} = P_{j,j-1} = 0.5.$$

and

$$P_{k,k-1} = 1 = P_{-k,-k+1}.$$

Example: Symmetric Random Walk

Let's see if this Markov chain is time-reversible. Can we find a vector q that satisfies the condition in the theorem?

What if

$$q_k P_{k,k-1} = q_{k-1} P_{k-1,k} \quad \text{Then}$$

$$q_k = q_{k-1} \times 0.5$$

and similarly,

$$q_{-k} = q_{1-k} \times 0.5.$$

For $|j| < k - 1$,

$$q_j P_{j,j+1} = q_{j+1} P_{j+1,j} \quad \text{or}$$

$$0.5q_j = 0.5q_{j+1}.$$

Therefore, all q 's other than the q_k and q_{-k} are equal, and have twice the value of q_k and q_{-k} .

Example: Symmetric Random Walk

$$q_j = \frac{1}{2(k-1) + 1 + 1} = \frac{1}{2k}$$

and

$$q_k = q_{-k} = \frac{1}{4k}$$

⇒ This Markov chain is time-reversible.

q is the steady-state distribution.

e.g. $k = 3$:

$$q_3 = q_{-3} = \frac{1}{12}$$

$$q_2 = q_1 = q_0 = q_{-1} = q_{-2} = \frac{1}{6}.$$

A Simulation Check on the Calculation

Entering the transition matrix:

```
P <- matrix(c(0, 1, 0, 0, 0, 0, 0,  
.5, 0, .5, 0, 0, 0, 0, 0, .5, 0, .5, 0, 0, 0,  
0, 0, .5, 0, .5, 0, 0, 0, 0, 0, 0, 0.5, 0, .5, 0,  
0, 0, 0, 0, .5, 0, .5, 0, 0, 0, 0, 0, 1, 0), nrow=7, byrow = TRUE)
```

A Simulation Check on the Calculation

Simulating the random walk:

```
Ntransitions <- 100000 # number of moves
location <- numeric(Ntransitions) #initializing
current.state <- 1 # initial stock
for (t in 1:Ntransitions) {
  current.state <- sample(1:7,
    size = 1, prob = P[current.state, ])
  location[t] <- current.state
}
pi <- table(location)/Ntransitions
pi
```



```
## location
##      1      2      3      4      5      6      7
## 0.08196 0.16490 0.16702 0.16906 0.16881 0.16604 0.08221
```

Conventional Calculation of the Steady-State Vector

```
A <- t(P) - diag(rep(1, 7))
```

```
A <- rbind(A, rep(1, 7))
```

```
RHS <- c(rep(0, 7), 1)
```

```
options(digits=3)
```

```
qr.solve(A, RHS)
```

```
## [1] 0.0833 0.1667 0.1667 0.1667 0.1667 0.1667 0.1667 0.0833
```

Other Time-Reversible Markov Chains

Suppose $\{\pi_i, i = 0, \pm 1, \pm 2, \dots\}$ is a set of positive real numbers with $\sum_{i=-\infty}^{\infty} \pi_i = 1$. (This is a probability distribution on the integers.)

Set

$$P_{i,j} = \frac{1}{6} \min \left(\frac{\pi_j}{\pi_i}, 1 \right), \text{ for } j = i - 2, i - 1, i + 1, i + 2$$

and 0 for $|j - i| > 2$. $P_{i,i}$ is set to ensure that the row sums of P are 1.

To verify that the Markov chain is reversible, show that

$$\pi_i P_{i,i+2} = \pi_{i+2} P_{i+2,i}$$

and so on.*

P is an example of an infinite-state time-reversible Markov chain. Note that the steady-state vector has infinite length and has i th entry π_i .

*This equation follows by noting that the left hand side is $\frac{1}{6} \min(\pi_j, \pi_i)$ and the right hand side is $\frac{1}{6} \min(\pi_i, \pi_j)$.

Simulating from the Infinite State Markov Chain

Example:

Suppose $\pi_i = k/(i+1)^4$ for $i > 0$ and $\pi_i = 0$ for all $i < 1$. k is a constant that ensures that $\sum_{i=1}^{\infty} \pi_i = 1$. Note that we can simulate from this Markov chain even without knowing k .

```
pi.fun <- function(i) {  
  out <- 0  
  if (i > 0) out <- 1/(i+1)^4  
  out  
}
```

Simulating from the Infinite State Markov Chain

```
Ntransitions <- 20000
X <- numeric(Ntransitions)
current.state <- 50 # initialize the Markov chain
for (n in 1:Ntransitions) {
  i <- current.state
  P <- c(min(pi.fun(i-2)/pi.fun(i), 1),
        min(pi.fun(i-1)/pi.fun(i), 1),
        min(pi.fun(i+1)/pi.fun(i), 1),
        min(pi.fun(i+2)/pi.fun(i), 1))/6
  P0 <- 1 - sum(P)
  P <- c(P[1:2], P0, P[3:4])
  transition <- sample(seq(-2,2,1), size = 1, prob = P)
  current.state <- current.state + transition
  X[n] <- current.state
}
observedDist <- table(X[-c(1:1000)])
```

Simulating from the Infinite State Markov Chain

```
observedDist
```

```
##
```

```
##      1      2      3      4      5      6      7      8
```

```
## 14662 2865 846 334 160 94 18 14
```

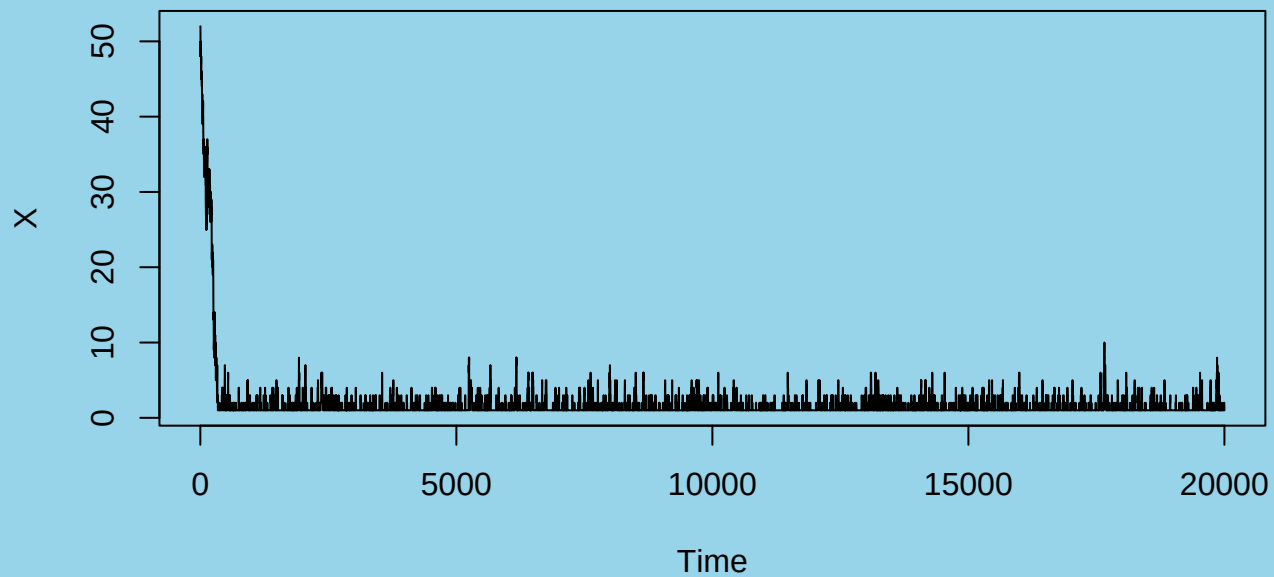
```
##      9     10
```

```
##      3      4
```

Burn-In

Why omit the first 1000 observations?

```
ts.plot(X)
```



Estimating k

$$\pi_2 = k/(2+1)^4 = k/81$$

so an estimate of k can be obtained by multiplying the observed probability of a 2 by 81:

```
k <- observedDist[2]/19000*81
```

```
k
```

```
##      2
```

```
## 12.2
```

Markov Chain Monte Carlo Simulation

This procedure is one version of MCMC – developed by Metropolis and Hastings.

Procedure:

- 1. Given a distribution π , known up to a proportionality constant (k), find a time-reversible Markov chain with π as the steady state vector.**
- 2. Simulate from that Markov chain.**
- 3. After simulating for a long enough period (burn-in), the observed states follow the steady state distribution, i.e. π .**

Markov Chain Monte Carlo Simulation

The Law of Large Numbers for regular Markov chains allows us to estimate quantities such as $E[X]$ and $E[g(X)]$ for given functions $g(x)$ by calculating

$$\frac{1}{N} \sum_{n=1}^N X_n \text{ and } \frac{1}{N} \sum_{n=1}^N g(X_n).$$

MCMC Application - Bayesian Statistics

Example:

Suppose N is Poisson distributed with mean 20, and given N , X is binomially distributed with parameters N and $p = 0.5$.

N is not observed, but suppose $X = 5$. Use MCMC to simulate the distribution of N , given X .

Terminology: the Poisson distribution for N is the *prior* distribution.

the distribution of N , given $X = 5$, is called the *posterior* distribution.

MCMC Application - Bayesian Statistics

```
posterior.fun <- function(i, x) {  
  out <- 0  
  if (i >= x) out <- dpois(i, lambda = 20) *  
    dbinom(x, size = i, prob = .5)  
  out  
}
```

```
pi.fun <- function(i) {  
  posterior.fun(i, x=5)  
}
```

Simulating the Markov Chain

```

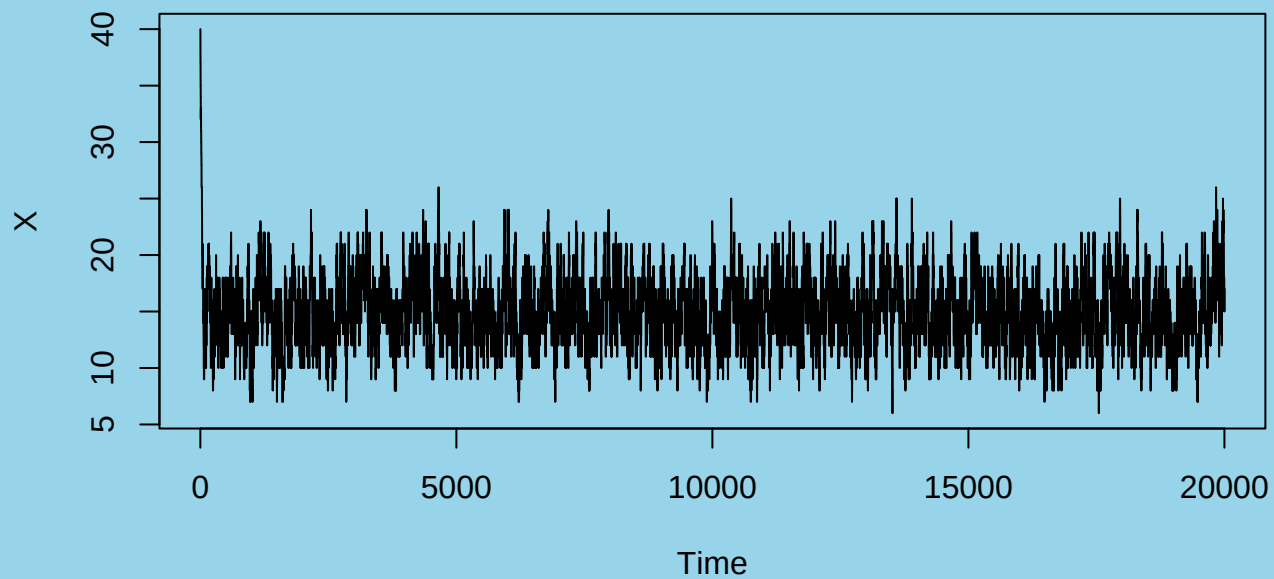
Ntransitions <- 20000
X <- numeric(Ntransitions)
current.state <- 40 # initialize the Markov chain
for (n in 1:Ntransitions) {
  i <- current.state
  P <- c(min(pi.fun(i-2)/pi.fun(i), 1),
         min(pi.fun(i-1)/pi.fun(i), 1),
         min(pi.fun(i+1)/pi.fun(i), 1),
         min(pi.fun(i+2)/pi.fun(i), 1))/6

  P0 <- 1 - sum(P)
  P <- c(P[1:2], P0, P[3:4])
  transition <- sample(seq(-2,2,1), size = 1, prob = P)
  current.state <- current.state + transition
  X[n] <- current.state
}
observedDist <- table(X[-c(1:1000)])

```

Plotting the Trace

```
ts.plot(X)
```



Posterior Distribution of N

```
options (width=50)
```

```
observedDist
```

```
##
```

```
##      6      7      8      9     10     11     12     13     14     15
```

```
##      9     51    140    413    782   1213   1747   2117   2299   2342
```

```
##     16     17     18     19     20     21     22     23     24     25
```

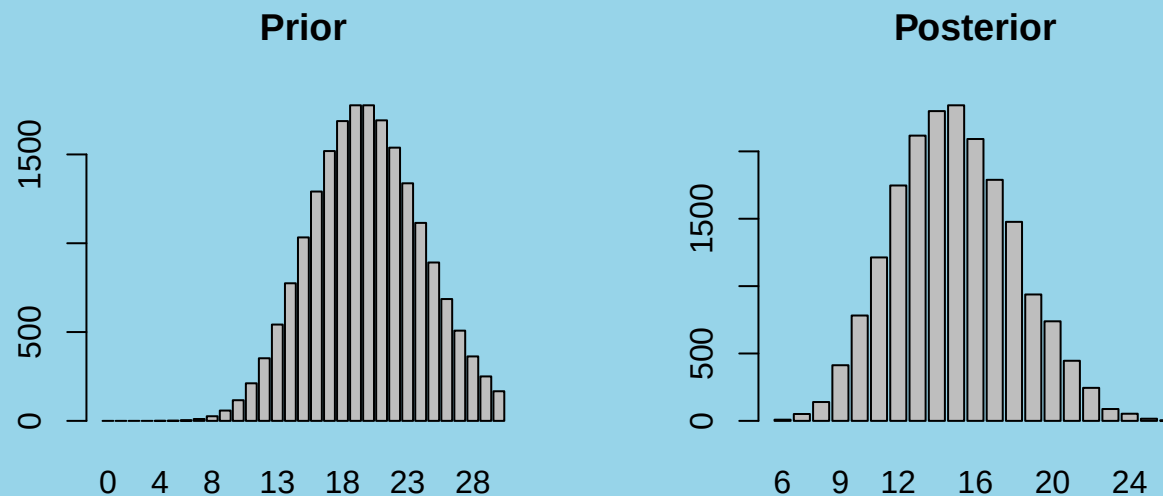
```
##  2092  1789  1477    938    739    446    245     88     53     16
```

```
##     26
```

```
##      4
```

Posterior Distribution of N

```
par(mfrow=c(1,2))
theoryDist <- 20000*dpois(0:30, lambda = 20)
names(theoryDist) <- 0:30
barplot(theoryDist, main = "Prior")
barplot(observedDist, main = "Posterior")
```



This is how the *data* $X = 5$ influences our *belief* (initially, Poisson(20)) about the distribution of the unknown value N .

What if our Prior Belief was Different?

e.g. $\lambda = 4$:

```
posterior.fun <- function(i, x) {  
  out <- 0  
  if (i >= x) out <- dpois(i, lambda = 4) *  
    dbinom(x, size = i, prob = .5)  
  out  
}
```

Simulating the Markov Chain

```

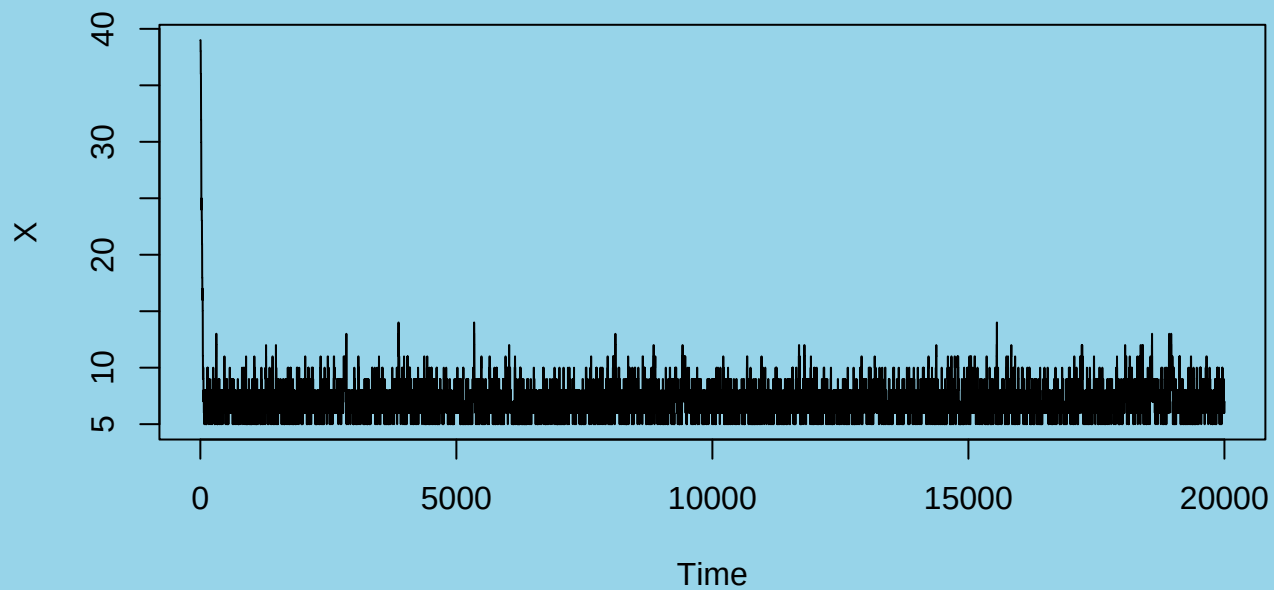
Ntransitions <- 20000
X <- numeric(Ntransitions)
current.state <- 40 # initialize the Markov chain
for (n in 1:Ntransitions) {
  i <- current.state
  P <- c(min(pi.fun(i-2)/pi.fun(i), 1),
         min(pi.fun(i-1)/pi.fun(i), 1),
         min(pi.fun(i+1)/pi.fun(i), 1),
         min(pi.fun(i+2)/pi.fun(i), 1))/6

  P0 <- 1 - sum(P)
  P <- c(P[1:2], P0, P[3:4])
  transition <- sample(seq(-2,2,1), size = 1, prob = P)
  current.state <- current.state + transition
  X[n] <- current.state
}
observedDist <- table(X[-c(1:1000)])

```

Plotting the Trace

```
ts.plot(X)
```



Posterior Distribution of N

```
options (width=50)
```

```
observedDist
```

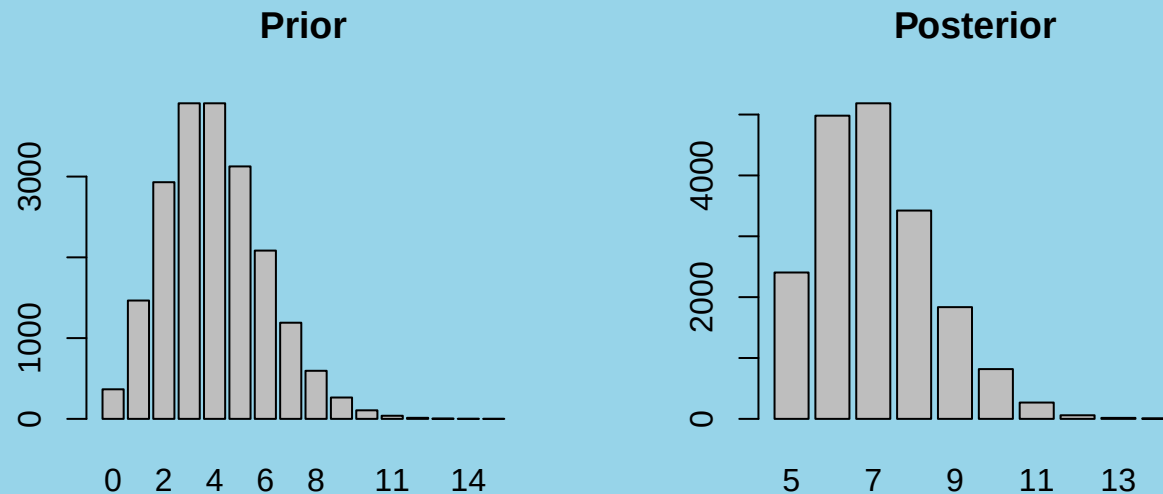
```
##
```

```
##      5      6      7      8      9     10     11     12     13     14
```

```
## 2405 4982 5185 3423 1836  818  268   59   16    8
```

Posterior Distribution of N

```
par(mfrow=c(1,2))
theoryDist <- 20000*dpois(0:15, lambda = 4)
names(theoryDist) <- 0:15
barplot(theoryDist, main = "Prior")
barplot(observedDist, main = "Posterior")
```



This is how the *data* $X = 5$ influences our *belief* (initially, $\text{Poisson}(4)$) about the distribution of the unknown value N .

Using Built-In Software

Perhaps the best way to do MCMC in R is with the `metrop()` function in C. Geyer's *mcmc* package:

```
metrop(obj, initial, nbatch, blen = 1, nspac = 1,  
        scale = 1, outfun, debug = FALSE, ...)
```

Main Arguments:

- **obj**: an R function which evaluates the unnormalized posterior distribution or the result of a previous call to this function.
- **initial**: the initial state of the Markov chain.
- **scale**: controls the proposal step size in the random walk used for the Markov chain.

Using Built-In Software

A simpler to use function, good for learning with, is `rwmetrop()` in J. Albert's *LearnBayes* package.

```
rwmetrop(logpost, proposal, start, m, ...)
```

Arguments:

- `logpost`: **function** defining the log posterior density
- `proposal`: **a list** containing `var`, an estimated variance-covariance matrix, and `scale`, the Metropolis scale factor
- `start`: **vector** containing the starting value of the parameter
- `m`: **the number of iterations of the chain**

Using Built-In Software - Random Walk Metropolis-Hastings

Example:

- Suppose M is a random variable having a normal distribution with parameter μ and standard deviation $\sigma_M = 0.1$.
- Suppose, conditional on M , X is normally distributed with mean M and with known variance σ^2 .
- We can use *Random Walk* Metropolis-Hastings to simulate from the posterior distribution of M , given the value or values of X .

Random Walk Metropolis-Hastings

- Random Walk Metropolis-Hastings is a version of the random walk where the jumps follow a continuous (actually, normal) distribution.
- The jump size is controlled by a single parameter: `scale`. Larger values of `scale` will give larger jumps.
- Too few jumps and too many large jumps will prevent the Markov chain from converging, so the scale needs to be chosen so that the proportion of *accepted proposals* is a moderate value, not too near 1% or 99%. Some people recommend 25%, but this is not always the best.
- An accepted proposal amounts to a transition in the Markov chain where a move is made; a rejected proposal is a transition where the Markov chain stays in the same state.

Simulating a data set

We simulate a sample of 10 observations from a population where the mean fluctuates around 3.6 according to a normal distribution with a standard deviation of 0.1:

```
M <- rnorm(1, mean = 3.6, sd = .1) # true prior distribution of M
```

This M value is not observed.

The observed data are normally distributed with mean M and a standard deviation of 1.0 (assumed known).

```
data <- rnorm(10, mean = M, sd = 1) # sample of size 10
data
## [1] 3.21 4.13 3.06 5.72 3.39 2.72 4.80 2.79 4.49
## [10] 4.08
```

Setting up the (log) prior

In practice, we do not know the prior distribution of M .

Here, we will suppose (correctly, but not expectedly) a normal distribution, and we will need to choose the mean μ_M and standard deviation σ_M somehow, perhaps from historic data.

```
logprior <- function(M, muM, sigmaM) {  
  dnorm(M, mean = muM, sd = sigmaM, log = TRUE)  
}
```

Setting up the (log) posterior

The log posterior is the sum of the log likelihood (of the data, conditional on the prior parameters) and the log prior

```
logNormNorm <- function(M, datapar) {  
  X <- datapar$data  
  muM <- datapar$par[1]  
  sigmaM <- datapar$par[2]  
  loglike <- 0  
  for (i in 1:length(X)) {  
    loglike <- loglike + (dnorm(X[i], mean = M,  
                                sd=1, log = TRUE))  
  }  
  logprior <- logprior(M, muM, sigmaM)  
  return(loglike + logprior)  
}
```

Additional Parameters for the MCMC Simulation

We arbitrarily choose μ_M and σ_M as 3.61 (very close to the truth) and 0.1 (the truth):

```
# data and prior parameters:
datapar <- list(data=data, par = c(muM = 3.61, sigmaM = .1))
library(LearnBayes) # load MCMC package
fit <- laplace(logNormNorm, 1, datapar)
fit

## $mode
## [1] 3.63
##
## $var
##      [,1]
## [1,] 0.00909
##
## $int
## [1] -13.8
##
## $converge
## [1] TRUE
```

We have also used the Laplace approximation to the posterior density here in order to help estimate some other parameters. In particular, `fit$var` gives an estimate of the variance-covariance matrix (1×1 in this case) of the parameter(s).

We can also use the mode value as our starting value for the Markov chain simulation.

Running the MCMC Simulation

```
start <- 3.63 # starting value for M in MCMC simulation
varcov <- matrix(fit$var, nrow=1) # estimated variance of M
proposal = list(var = varcov, scale = 1)
m <- 10000 # number of Markov chain simulation steps
s <- rwmetrop(logNormNorm, proposal, start, m, datapar = datapar)
```

Did We Choose the Right Scale?

```
s$accept
## [1] 0.699
```

Not good.

Increasing the scale in order to decrease the acceptance rate

```
proposal = list(var = varcov, scale = 3)
s <- rwmetrop(logNormNorm, proposal, start, m, datapar = datapar)
```

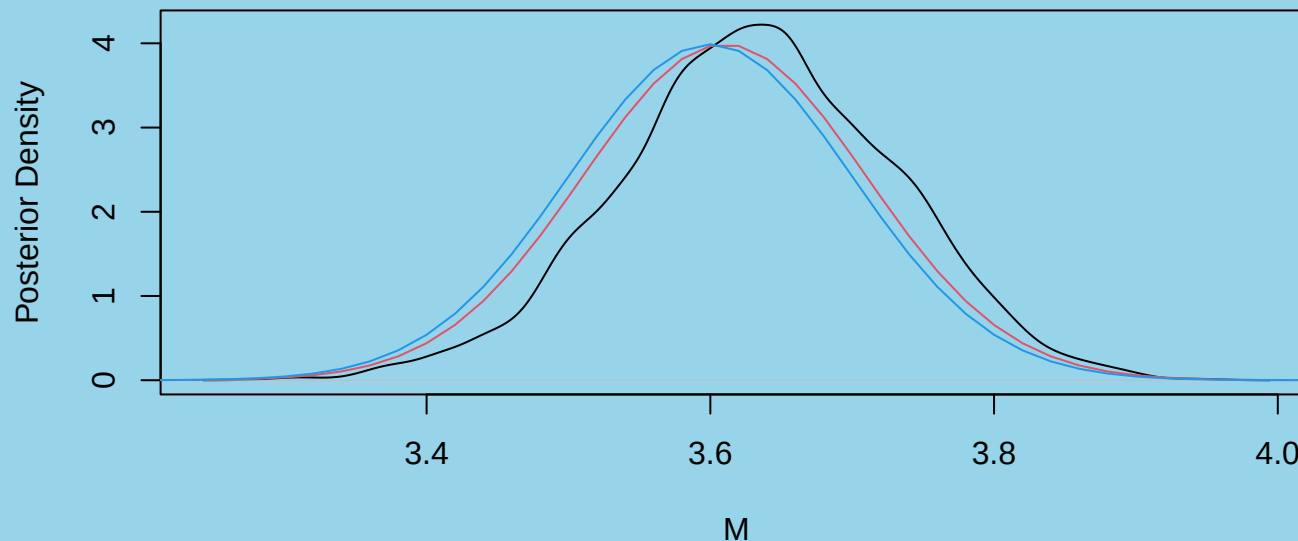
Did We Choose the Right Scale?

```
s$accept
## [1] 0.373
```

Not bad.

Visualizing the Posterior Distribution

```
par(mar=c(4, 4, 1, 1))
plot(density(s$par), xlab="M", ylab="Posterior Density", main = " ")
curve(dnorm(x, mean = datapar$par[1], sd = datapar$par[2]), 3, 5,
      col=2, add=TRUE)
curve(dnorm(x, mean = 3.6, sd = .1), 3, 5,
      col=4, add=TRUE)
```



the prior distribution is plotted in red.

the true distribution is plotted in blue.

All three distributions are similar, because the prior was chosen almost perfectly here.

A More Realistic Scenario

We would be more likely to choose μ_M and σ_M incorrectly, say, as 4.0 (still close to the truth) and 0.5 (more spread out than the truth):

```
# data and prior parameters:
datapar <- list(data=data, par = c(muM = 4.0, sigmaM = .5))
library(LearnBayes) # load MCMC package
fit <- laplace(logNormNorm, 1, datapar)
fit

## $mode
## [1] 3.88
##
## $var
##      [,1]
## [1,] 0.0714
##
## $int
## [1] -14.1
##
## $converge
## [1] TRUE
```

We have again used the Laplace approximation to the posterior density here in order to help estimate some other parameters. In particular, `fit$var` gives an estimate of the variance-covariance matrix (1×1 in this case) of the parameter(s).

We can also use the mode value as our starting value for the Markov chain simulation.

Running the MCMC Simulation

```
start <- 3.88 # starting value for M in MCMC simulation
varcov <- matrix(fit$var, nrow=1) # estimated variance of M
proposal = list(var = varcov, scale = 3)
m <- 10000 # number of Markov chain simulation steps
s <- rwmetrop(logNormNorm, proposal, start, m, datapar = datapar)
```

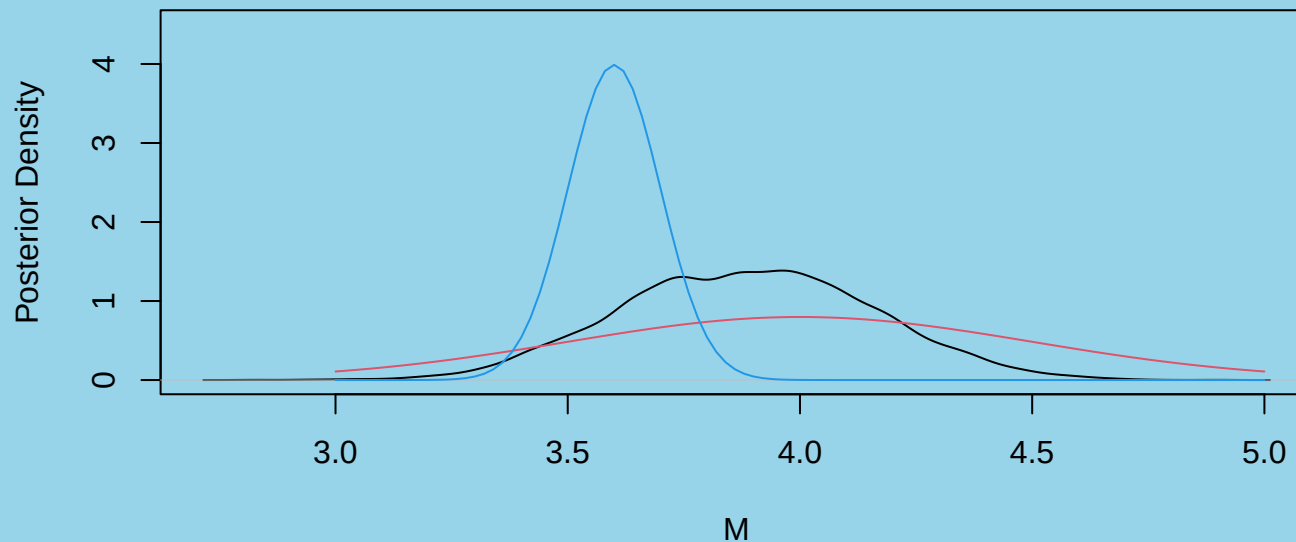
Did We Choose the Right Scale?

```
s$accept
## [1] 0.377
```

Okay.

Visualizing the Posterior Distribution

```
par(mar=c(4, 4, 1, 1))
plot(density(s$par), xlab="M", ylab="Posterior Density", main = " ",
ylim = c(0, 4.5))
curve(dnorm(x, mean = datapar$par[1], sd = datapar$par[2]), 3, 5,
      col=2, add=TRUE)
curve(dnorm(x, mean = 3.6, sd = .1), 3, 5,
      col=4, add=TRUE)
```



the prior distribution is plotted in red.

the true distribution is plotted in blue.

Now, the prior has pulled the posterior away from the truth. Or more accurately, the data have pulled the posterior toward the truth.

What if we have more data?

If we have more data, the effect of the prior should be reduced.

We now pretend that we have 100 data points, instead of only 10.

```
#M <- rnorm(100, mean = 3.6, sd = .1) # true prior distribution of M
data <- rnorm(100, mean = M, sd = 1) # sample of size 100
datapar$data <- data
```

Running the MCMC Simulation

We could run the Laplace approximation again, but we will just leave everything as before. The only change is the larger sample, and a reduced scale so that the acceptance rate in the MCMC sampler is satisfactory.

```
start <- 3.88 # starting value for M in MCMC simulation
varcov <- matrix(fit$var, nrow=1) # estimated variance of M
proposal = list(var = varcov, scale = 1)
m <- 10000 # number of Markov chain simulation steps
s <- rwmetrop(logNormNorm, proposal, start, m, datapar = datapar)
```

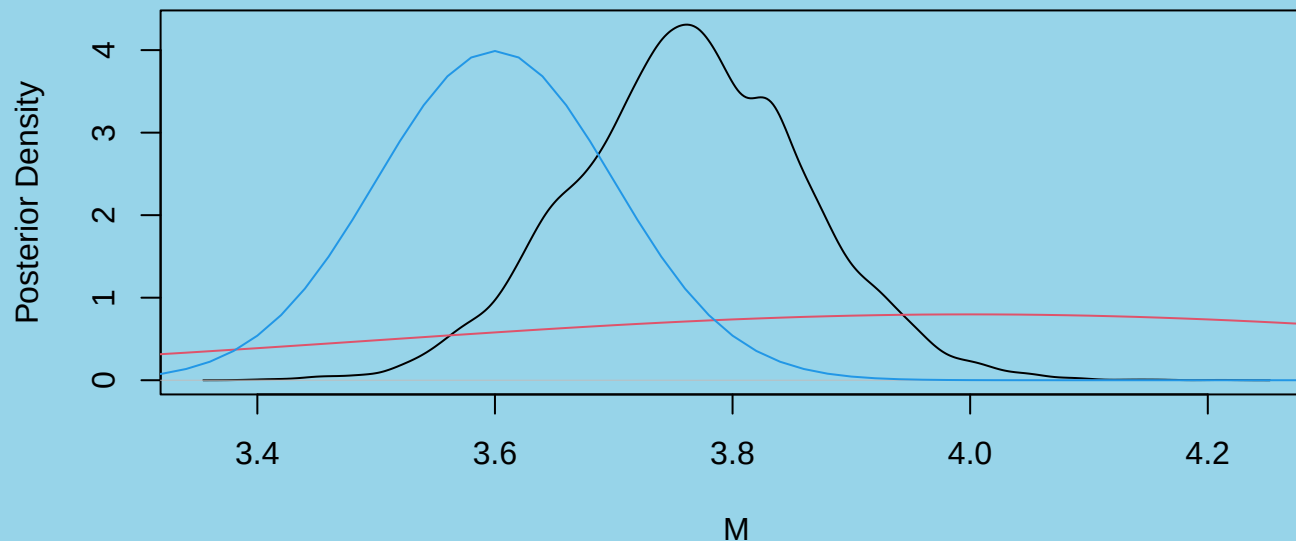
Did We Choose the Right Scale?

```
s$accept
## [1] 0.402
```

Okay.

Visualizing the Posterior Distribution

```
par(mar=c(4, 4, 1, 1))
plot(density(s$par), xlab="M", ylab="Posterior Density", main = " ")
curve(dnorm(x, mean = datapar$par[1], sd = datapar$par[2]), 3, 5,
      col=2, add=TRUE)
curve(dnorm(x, mean = 3.6, sd = .1), 3, 5,
      col=4, add=TRUE)
```



the prior distribution is plotted in red.

the true distribution is plotted in blue.

Now, the data have really pulled the posterior toward the truth.

Comparing the Bayesian Approach with the Frequentist

Bayes:

```
mean(s$par)
```

```
## [1] 3.76
```

```
sd(s$par)
```

```
## [1] 0.0979
```

We can use this to find an interval that contains the true mean with probability 0.95.

Frequentist:

```
mean(data)
```

```
## [1] 3.76
```

```
sd(data) / sqrt(length(data))
```

```
## [1] 0.0883
```

We can use this to find a 95% confidence interval.
